Construção gulosa randomizada e busca tabu para a localização de locais de prova

Bruno Keil[1]

[1]Departamento de Ciência da Computação – Universidade Federal de Juiz de Fora
DCC136 – Inteligência Computacional – 2026.3

Abstract. *We study candidate assignment and exam-type selection at capacitated schools. A randomized greedy constructor with augmenting-path repair is followed by variable neighborhood descent or tabu search. A small integer model certifies maximum attendance, while independent validation recomputes every reported solution. Experiments compare both methods and neighborhood, randomness and tenure variants over four instances and two limits on exam types per school.*

Resumo. *Estudamos a alocação de candidatos e a escolha dos tipos de prova oferecidos por escolas com capacidade limitada. Uma construção gulosa randomizada com reparação por caminhos aumentantes é seguida de descida em vizinhanças variáveis ou busca tabu. Um modelo inteiro pequeno certifica o atendimento máximo; a distância é otimizada heuristicamente. Os experimentos abrangem quatro instâncias, dois limites de tipos por escola e 30 sementes por configuração. Um validador independente recompõe cada solução inicial e final.*

1. Problema e hipóteses

O planejamento de locais de prova combina decisões de localização e alocação. A referência acadêmica é a aplicação brasileira estudada por Oliveira (2021), sob orientação de Rafael Martinelli Pinto e coorientação de Bruno Milanez [Oliveira 2021]. Neste trabalho, as escolas não possuem tipos previamente determinados: a otimização escolhe quais tipos hospedar e onde alocar cada candidato.

Sejam C os candidatos, L as escolas e T os tipos de prova. Cada candidato i tem tipo t_i e distância d_{ij} até cada escola j. A capacidade Q_j é a soma das capacidades de suas salas. Uma escola pode hospedar até $K \in \{1, 2\}$ tipos. Cada candidato deve ser alocado exatamente uma vez a uma escola compatível, sem ultrapassar a capacidade. O objetivo principal é minimizar $D = \sum_i d_{i,a_i}$, sem custos de abertura.

Adotamos capacidade agregada por escola. Não impomos um único tipo por sala; essa variante exigiria um modelo diferente. CEPs são strings; identificadores são preservados, mas índices seguem a ordem do arquivo. Latitude e longitude são lidas e validadas, sem uso no objetivo. As distâncias da matriz são armazenadas exatamente em milionésimos de km, aceitando até seis casas decimais. O limite $K \in \{1, 2\}$ é um parâmetro obrigatório de execução (`-tipos-por-escola`).

2. Representação e verificação de viabilidade

A solução contém o conjunto $\mathrm{typeSlots}_j$, com cardinalidade no máximo K, e o vetor a_i de escolas. O valor -1 representa candidato não alocado. Mantemos incremental-

mente a carga por escola, a carga por tipo, distância, excesso e quantidade de não alocados/incompatíveis. Escola utilizada significa carga positiva; um tipo habilitado pode não receber candidatos.

Capacidade total suficiente não garante viabilidade. Para certificar o máximo atendimento, usamos variáveis binárias z_{jt} e inteiras $x_{jt} \geq 0$, resolvendo:

$$\max \quad \sum_{j,t} x_{jt} \tag{1}$$

$$\text{sujeito a} \quad \sum_{j} x_{jt} \leq d_t \qquad \forall t, \tag{2}$$

$$\sum_{t} x_{jt} \leq Q_j \qquad \forall j, \tag{3}$$

$$x_{jt} \leq Q_j z_{jt}, \quad \sum_{t} z_{jt} \leq K \qquad \forall j, t. \tag{4}$$

Aqui d_t é a demanda do tipo t. Como não há restrições individuais de elegibilidade além do tipo, essas contagens inteiras bastam para construir uma alocação. O ótimo C^* determina o mínimo $n - C^*$ de não alocados. A pré-checagem usa SciPy/HiGHS [SciPy]; timeout é inconclusivo, nunca uma prova de inviabilidade. A atribuição de tipos obtida serve também como recurso de segurança da construção; portanto, o método tem apoio explícito de um modelo inteiro, cujo tempo é medido separadamente.

Tabela 1. Viabilidade comprovada no modelo agregado.

Instância	Candidatos	Escolas	Capacidade	Não aloc. K=1	K=2
Modelo	109	5	121	18	0
Instância 2	2550	10	3000	20	0
A	4518	15	4830	0	0
B	4442	15	5260	0	0

Para tipos fixos, usamos fluxo máximo em uma rede com fonte, tipos, escolas e sorvedouro. Capacidades são d_t, n nos arcos de compatibilidade e Q_j. A rede compartilha corretamente a capacidade entre dois tipos de uma escola.

3. Construção gulosa randomizada

Na primeira fase, começamos sem tipos habilitados. Para cada adição admissível (j, t), calculamos o aumento g_{jt} no fluxo máximo. Seja $q = \min(Q_j, d_t)$ e a_{jt} a média das q menores distâncias dos candidatos do tipo t à escola j, normalizada por $d_{\max}$. O critério é maximizar

$$\text{score}(j, t) = \frac{g_{jt}}{d_t} + \frac{g_{jt}}{Q_j} - a_{jt}.$$

Assim combinamos cobertura relativa, aproveitamento de capacidade e proximidade. Consideramos apenas aumentos positivos e escolas com menos de K tipos. Uma lista restrita usa o limiar de score $s_{\max} - \alpha(s_{\max} - s_{\min})$. Para $\alpha = 0$, escolhemos o primeiro máximo na ordem escola/tipo do arquivo; para $\alpha > 0$, sorteamos uniformemente.

Repetimos até cobrir C^* ou alcançar um impasse. Quando a fase gulosa estagna abaixo de C^* (por não desfazer escolhas iniciais em $K = 1$), aplica-se uma busca local de

reparo sobre os tipos (adicionando, substituindo ou permutando tipos entre escolas para maximizar o fluxo de atendimento e, em empate, reduzir a soma de proximidades a_{jt}, com perturbação em platôs). São permitidas até quatro tentativas (uma em $\alpha = 0$); persistindo o impasse, usam-se os tipos certificados como salvaguarda. O número de fallbacks é registrado.

Na segunda fase, escolhemos o candidato de maior arrependimento: diferença entre as distâncias à segunda e à primeira escola habilitada com vaga. Uma única opção recebe prioridade infinita; empates determinísticos usam a ordem de arquivo. A mesma regra de lista restrita controla a randomização. O candidato é alocado à opção mais próxima; opções afetadas são atualizadas quando uma escola lota.

Candidatos sem vaga são reparados por busca em largura de caminhos aumentantes. A cadeia desloca candidatos de escolas cheias até alcançar uma vaga, inclusive ejetando outro tipo quando $K = 2$. Cada cadeia aumenta o atendimento em um. A ausência de caminho certifica atendimento máximo para esses tipos; a fase anterior assegura que esse máximo é C^*. A especificação completa e os desempates estão em `docs/algoritmos.md`.

4. Avaliação e tratamento de violações

A função é

$$f(s) = D(s) + \lambda \sum_j \max(0, \text{carga}_j - Q_j) + \mu\big(U(s) + I(s)\big),$$

onde U e I contam candidatos não alocados e incompatíveis. Inicialmente $\lambda = \mu = B + 1$ km, com $B = nd_{\max}$. Como $0 \le D \le B$, uma unidade de violação domina qualquer diferença de distância. A busca mantém compatibilidade e limite de tipos como invariantes, mas a tabu permite excesso temporário e destino -1.

Após cinco estados viáveis consecutivos, dividimos λ por dois; após cinco inviáveis, dobramos seu valor. Limitamos λ entre $\max(10^{-6}\text{ km}, d_{\max}/10)$ e $16\lambda_0$; μ fica fixo. Um arquivo separado guarda apenas soluções sem excesso e sem incompatibilidade, primeiro minimizando não alocados e depois distância. Como a construção atinge C^*, a solução final é viável sempre que possível; nos casos impossíveis, possui o mínimo comprovado de não alocados. Não há prova de ótimo da distância.

5. Vizinhanças, VND e busca tabu

N1 – realocação. Mover um candidato às p escolas compatíveis mais próximas, distintas da atual. A tabu também permite removê-lo temporariamente para -1. Distância e excesso são avaliados por delta em tempo constante.

N2 – troca. Trocar escolas de dois candidatos do mesmo tipo, inclusive entre alocado e não alocado. O delta é $d_{ib} + d_{ha} - d_{ia} - d_{hb}$. Para evitar todos os pares de candidatos, mantemos ganhos por par de escolas e tipo: o melhor por arco no VND e os quatro melhores na tabu, para oferecer alternativas aos movimentos proibidos.

N3 – alteração de tipos. Adicionar/substituir tipo ou trocar tipos entre escolas. Filtramos propostas incapazes de manter C^* por fluxo. Liberamos candidatos das escolas

alteradas e os realocamos por regret e reparação; caminhos de reparo podem deslocar outros candidatos. A enumeração é amostrada e limitada por orçamento de reconstruções.

O VND visita N1, N2 e N3, reiniciando após melhoria. Compara primeira e melhor melhoria no conjunto efetivamente examinado. A busca tabu examina N1/N2 a cada iteração e N3 periodicamente ou por estagnação. Escolhe o menor delta admissível, inclusive positivo. Os atributos proibidos são candidato/escola deixada, par não ordenado de candidatos e escola/tipo removido. Tenure é fixo ou uniforme em $[t_{\min}, t_{\max}]$. Aspiração aceita tabu somente se produzir a primeira solução viável ou melhorar a melhor viável conhecida.

O encerramento ocorre por limite de iterações, estagnação, tempo opcional ou falta de movimento admissível. A semente torna decisões reproduzíveis no mesmo ambiente com orçamento de iterações; limites temporais podem mudar a trajetória. N3 amostrada e listas restritas impedem afirmar ótimo local sobre as vizinhanças completas.

6. Exemplo verificável e caso de falha

A instância `exemplo8.txt` tem oito candidatos, dois tipos e escolas de capacidades 3, 3 e 2. A construção escolhe E1/M2, E2/M1 e E3/M1,M2. C6 fica sem vaga; a cadeia C1→E2 e C6→E3 resolve o impasse. A solução inicial soma 49 km. Trocar C5 de E1 com C6 de E3 produz

$$\Delta = 6 + 9 - 10 - 12 = -7 \text{ km},$$

resultando em 42 km, redução de 14,29%. O trace, a matriz e todas as decisões estão em `docs/exemplo8.md`. A enumeração completa confirma 42 km para esses tipos fixos, não para todas as escolhas possíveis de tipos.

O Modelo com $K = 1$ possui seis tipos positivos e apenas cinco escolas: é impossível atender todos, embora a capacidade total seja suficiente. O máximo inteiro é 91/109. A instância 2 também é impossível em $K = 1$, com máximo 2530/2550. A instância A é viável, mas usa 93,54% da capacidade, dificultando alterações de tipos e reparações. Esses casos ilustram limites estruturais e heurísticos distintos.

7. Experimentos e resultados

Executamos 11 configurações para cada uma das quatro instâncias e cada K, com 30 sementes (1 a 30): 2640 execuções. As ablações incluem $\alpha = 0$, 0,3 e 0,7; primeira/melhor melhoria; N1/N2/N3 isoladas; VND/tabu; tenure 3, 10, $[5, 12]$ e $[10, 20]$. O orçamento é de 150 iterações, 50 sem melhoria, $p = 5$, N3 a cada 15 iterações e quatro reconstruções por chamada. O orçamento comum de iterações não implica mesmo número de avaliações ou mesmo tempo; registramos todos esses indicadores.

A implementação usa C++17, Apple Clang 21 com `-O2`, em macOS arm64. A pré-checagem usa SciPy 1.11.3 e NumPy 1.24.2. O manifesto registra versões, plataforma, sementes e hashes. Tempos são indicativos de uma máquina compartilhada, sem isolamento de CPU; separam construção, busca e pré-checagem. O tempo do ILP é compartilhado pelas 30 execuções de uma configuração.

O validador independente em Python relê cada instância e recompõe distâncias com `Decimal`, capacidades, tipos e identificadores. Todas as soluções iniciais e finais

são verificadas antes da inclusão nos resultados. Testes adicionais cobrem arquivos malformados, deltas versus recomputação, compartilhamento de capacidade, certificados e reprodução por semente.

Tabela 2. Resultados com $\alpha = 0{,}3$: 30 sementes por linha. Distâncias em km. Asterisco indica instância inviável; a distância é parcial.

Inst.	K	Método	Inicial média	Final média ± dp	Mín.	Máx.	Busca (s)
Modelo*	1	Tabu	359,51	319,35 ± 0,42	319,27	321,56	0.0011
Modelo*	1	VND	359,51	325,15 ± 9,92	319,27	361,41	0.0002
Modelo	2	Tabu	380,13	323,07 ± 15,43	298,08	372,60	0.0024
Modelo	2	VND	380,13	371,19 ± 23,32	337,71	407,36	0.0001
Inst. 2*	1	Tabu	12992,74	11817,25 ± 128,72	11646,20	12110,73	0.0525
Inst. 2*	1	VND	12992,74	12769,80 ± 716,08	11695,52	13955,52	0.0109
Inst. 2	2	Tabu	10883,47	10009,58 ± 2552,77	6447,61	14268,84	0.0653
Inst. 2	2	VND	10883,47	10710,26 ± 2291,35	6571,76	14388,72	0.0175
A	1	Tabu	30517,89	29473,64 ± 1102,77	27781,97	32933,12	0.1309
A	1	VND	30517,89	30204,54 ± 1217,13	28072,65	33693,26	0.0443
A	2	Tabu	29826,65	28732,02 ± 801,47	27232,55	30181,57	0.2104
A	2	VND	29826,65	29341,76 ± 941,61	27408,68	31813,05	0.0602
B	1	Tabu	18081,28	16947,05 ± 1120,92	15091,11	19613,50	0.1241
B	1	VND	18081,28	17892,07 ± 1370,75	15317,95	20451,39	0.0421
B	2	Tabu	18135,71	17283,04 ± 1073,33	15162,57	19237,14	0.1721
B	2	VND	18135,71	17842,85 ± 1182,81	16046,01	21118,85	0.0520

Tabela 3. Ablações em A, K=2. As outras instâncias constam no CSV completo.

Configuração	Média	dp	Mínimo	Máximo	Busca (s)
n1	29815,67	976,44	28119,96	32481,26	0.0026
n2	29096,15	912,77	27151,58	31253,00	0.0247
n3	29815,98	971,54	28119,96	32481,26	0.0057
tabu_a03	28732,02	801,47	27232,55	30181,57	0.2104
tabu_t10	28890,98	933,06	27232,55	30721,09	0.2159
tabu_t10_20	28926,65	850,67	27232,55	30721,16	0.1990
tabu_t3	28891,33	933,06	27232,55	30721,03	0.2150
vnd_a0	30008,24	0,00	30008,24	30008,24	0.0636
vnd_a03	29341,76	941,61	27408,68	31813,05	0.0602
vnd_a07	29703,58	1293,29	27560,18	32873,86	0.0627
vnd_first	29675,95	988,27	28006,45	32445,47	0.0565

No orçamento utilizado, a tabu com $\alpha = 0{,}3$ apresentou menor distância média que o VND correspondente em 8 dos oito pares instância/K, incluindo casos de atendimento parcial. Em A, K=1, a média passou de 30204,54 km no VND para 29473,64 km na tabu, com tempos médios de busca de 0.044 s e 0.131 s. Essa comparação não estabelece significância estatística e usa o mesmo limite de iterações, não o mesmo tempo.

Em A, K=1, o reparo local de tipos na construção atingiu o atendimento máximo sem recorrer aos tipos do certificado (0 fallbacks nas 30 sementes com $\alpha = 0{,}3$). Para A, K=2, a configuração de menor média entre as testadas foi `tabu_a03`, com 28732,02 km. Não há uma tenure universalmente recomendada a partir desses dados.

Muitas execuções atingiram 150 iterações; portanto, parte das diferenças reflete também a velocidade de melhoria antes do limite. A flexibilização para K=2 amplia o conjunto viável, mas uma heurística com orçamento finito pode encontrar resultado pior que em K=1, como observado em algumas médias de B. Essa é uma limitação de exploração, não uma inversão da relação entre ótimos dos modelos.

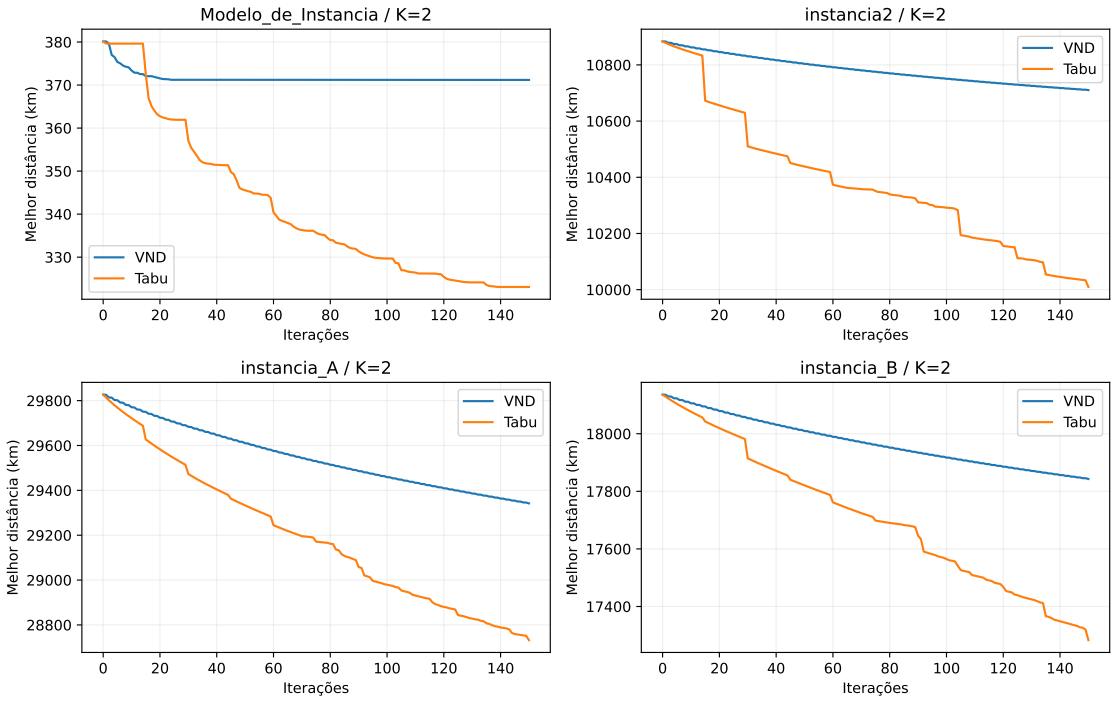


Figura 1. Média da melhor distância por iteração em K=2. Trajetórias encerradas são mantidas constantes até o maior horizonte observado.

Em uma execução adicional de A, K=1, com semente 1, $\alpha = 0{,}3$ e limite de 5000 iterações, o VND encerrou após 2740 iterações, em 1.012 s de busca, passando de 29542,03 para 27323,54 km. Essa medição individual ilustra o alvo de desempenho, não substitui as 30 repetições da campanha e continua sujeita à amostragem de N3.

As tabelas completas estão em `resultados/resumo.csv`, com mínimo, média, desvio padrão amostral e máximo; `execucoes.csv` registra cada execução. Os JSONs contêm as soluções, distribuição por escola/tipo, ocupações e curvas. Os casos inviáveis permanecem identificados: sua distância é parcial e não deve ser comparada diretamente a soluções completas. A análise é descritiva; não afirmamos significância estatística ou superioridade universal de uma configuração.

8. Conclusões e continuidade

O projeto passou a otimizar tipos e alocações, preservando os componentes úteis da implementação inicial: vetor de atribuições, regret, deltas, ganhos por pares e transporte por caminhos aumentantes. A certificação separa falha heurística de inviabilidade estrutural; a reparação assegura o atendimento certificado. A busca local explora tanto alocações quanto tipos, com controle explícito de custo computacional.

As principais limitações são a dependência do certificado inteiro, a vizinhança N3 amostrada, as listas restritas e o orçamento experimental. Como continuidade, propomos listas adaptativas, maior exploração dos tipos, avaliação de sensibilidade de p e estudo da variante de capacidade por sala. Uma referência de transporte com tipos fixos não constitui limitante global do problema completo.

9. Reprodutibilidade e disponibilidade

O README detalha dependências e parâmetros. Os comandos principais são:

```
make
make test
python3 scripts/experimentos.py
```

O histórico foi preservado em bundle Git devido à restrição de escrita em `.git` no ambiente de desenvolvimento. Publicação no GitHub pendente de autenticação e definição do repositório.

Referências

Oliveira, T. E. (2021). *Solving a University Admission Exam Location Problem: an Application in Brazil*. Dissertação de Mestrado, Departamento de Engenharia Industrial, PUC-Rio. Orientador: Rafael Martinelli Pinto; coorientador: Bruno Milanez. `https://www.maxwell.vrac.puc-rio.br/52465/52465.PDF`.

SciPy. *scipy.optimize.milp: Mixed-integer linear programming*. Documentação da interface usada para a pré-checagem. `https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html`.